Product Requirements

Polycarpic — a virtual Single Family Office

Owner: `product-manager` agent

Status: Approved 2026-09-27 (Research gate) — v1, first initiative (`POLY-V1`) **Version:** 1.0

Last updated: 2026-09-27

CONTENTS

1. Overview & Problem Statement
    1. High-Level Approach
2. Goals & Success Metrics
3. Non-Goals
4. User Stories & Use Cases
5. Functional Requirements
6. Non-Functional Requirements
7. Design Considerations
8. Technical Considerations
9. Timeline & Milestones
10. Open Questions & Risks
11. Appendix
    1. FAQs

1. Overview & Problem Statement

Why we're building this. The user problem, who feels it, and the desired business outcome.

Problem. The Principal manages personal and family finances — budgeting, investment tracking, and quarterly tax estimation — across multiple legal entities (Person, Trust, Business, Household) using disconnected spreadsheets and per-provider dashboards. There is no single, continuously updated system of record that shows true financial health per entity, categorizes cash flows by tax treatment, tracks investment lots with accurate book and market value, or reliably estimates quarterly taxes.

Who feels it. The Principal today, as owner/operator across several entities. The Principal's spouse and other household members feel it too, once shared-access (membership) features ship — they co-own jointly-titled accounts and need their own view into household finances without owning the underlying ledger.

Status quo and its cost. Manual spreadsheets and per-provider dashboards (bank and brokerage sites) require hand re-entry and reconciliation — the reason the Principal wants this to grow into a virtual **Single Family Office** rather than continuing to patch spreadsheets together. A related prior project, mosko-fintech, explored some of this ground but is explicitly not the blueprint: this project starts from what the Principal wants and doesn't want (kickoff §3), not from that codebase's structure.

Business outcome. A self-hosted system of record with a genuine double-entry general ledger, automated provider-agnostic transaction import, and per-entity dashboards — replacing manual reconciliation with always-current visibility into net worth, cash flow, and tax position across every entity the Principal controls. Built so any self-hoster can run their own instance, though the Principal will never operate it as a hosted service for anyone else — a standing decision, not a v1-only deferral (§3).

1.1 High-Level Approach

The rough shape of how we'll tackle the problem — a sentence or two so a reader can squint and see the same solution. Not a design or architecture; just the framing. (e.g. for "feature discoverability" → "an in-app notification center for relevant new features".)

A modular, layered SvelteKit application on a genuine double-entry general ledger in PostgreSQL under row-level security. Transactions arrive through provider-agnostic import adapters (a deterministic fake source first, then SimpleFIN, Plaid, and direct broker APIs), land as immutable posted journal entries once confirmed, and surface through Monarch/Origin-style per-entity dashboards. Delivered incrementally by initiative — this PRD scopes the first initiative (`POLY-V1`);

later initiatives (stock research, Monte Carlo modelling, a consolidated household dashboard, multi-currency processing, etc.) are explicit Non-Goals for now (§3). Detail belongs in ARCH.

2. Goals & Success Metrics

What measurable outcomes prove the product succeeded? Mix measurable (metrics) and immeasurable (feelings). Include counter-metrics where relevant. Ideally in priority order.

1. **Replace spreadsheets as the system of record.** Metric: by day 90 after POLY-V1 ships, 100% of the Principal's transactions for at least one entity flow through import → draft → confirm, with zero manual spreadsheet entries for that entity — provisional. Counter-metric: the manual-correction/reversal rate stays low enough that reconciling in-app isn't more work than the prior spreadsheet — numeric threshold TBD.
2. **Quarterly estimated-tax payments land right.** Metric: quarterly transfers into the prepaid-tax accounts (§5) match the app's guiding figure, and at filing there is no underpayment penalty. Refund/balance-due tolerance is set after the first full tax year from observed data. (Comparing to a manual calculation was dropped — filing stays in tax software, and the comparison measured nothing.)
3. **Cash in Transit stays fully explained.** Metric: every open item in the *Cash in Transit* clearing account is a known in-flight transfer at each check, with zero items older than a configurable aging threshold (default 3 business days) — aged items surface on an exceptions report instead.
4. **Self-hosting cost stays low** — a guardrail, not a growth goal. Counter-metric: monthly third-party spend stays near zero beyond the VPS and any data providers actually in use (Plaid trial items, Resend). No dollar figure set yet.

3. Non-Goals

Explicitly out of scope — and why. Non-goals are as clarifying as goals: they shut down ambiguity, prevent scope creep, and make trade-offs visible.

1. Stock research and valuation engine for screening — deferred; not needed until the core ledger and dashboards exist.
2. Monte-Carlo portfolio survivability and efficient-frontier modelling — deferred; depends on real historical data already flowing through the ledger.
3. Capitalization table per entity — deferred; only relevant for entity types that issue units, none of which are in the first initiative.
4. Business-type features (literal inventory management, etc.) — deferred; the first initiative targets Person/Trust/Household budgeting and investing, not business operations.
5. Consolidated household dashboard (the roll-up view) — the underlying schema (parent link + membership) ships in the first initiative, but the view itself is a later milestone.
6. Multi-currency process (rate feed, period-end revaluation, FX reports) — schema-ready (every journal line already carries currency + native/base amounts) but the process is deferred until foreign currency is actually held again.
7. User-authored report scripting — deferred until the declarative report definitions (§5) prove insufficient for a named report.
8. Period locking of closed books — deferred; the posting-date check is designed so locking can be added later without a schema change.
9. An MCP server exposing a user's data to their own assistant — feasible later as a thin client of the same API, authenticated with the user's own token; out of scope until the API exists.
10. A hosted, multi-user offering run by the Principal — a **standing** non-goal, not a v1-only deferral: the app is built to be self-hostable by anyone, but the Principal will never operate it as a service for others. If that decision is ever deliberately reversed, GLBA Safeguards Rule, state financial-privacy law, and PCI DSS (if card data is ever touched directly) would likely apply (SecEng read, `temp/seceng-research-notes.md`).

4. User Stories & Use Cases

"As a <persona>, I want to <action> so that <outcome>." Aim for 3–8 stories on v1.

1. As the Principal, I want a dashboard of financial health per entity (Person, Trust, Business, Household), so that I can tell at a glance how each entity is doing without opening separate spreadsheets or provider sites.
2. As the Principal, I want my bank and brokerage transactions imported automatically and staged as drafts, so that I can review and confirm them instead of hand-entering every transaction.
3. As the Principal, I want categorized expense and revenue cash flows tagged by tax treatment, so that I can distinguish tax-consequence expenses from owner draws/distributions when tax season comes.
4. As the Principal, I want a continuously updated view, per entity and jurisdiction, of my pending tax liability, the required safe-harbor payment, and the gap between them, so that I know exactly how much to wire by each due date without doing the calculation by hand.
5. As the Principal, I want lot-based tracking of my marketable securities with both GL book value and daily market value, so that I can see accurate cost basis alongside current net asset value.
6. As the Principal, I want a balance sheet by GL account and a separate one by custodial (physical holding) account, so that I can reconcile what the ledger says against what my custodians report.
7. As the Principal, I want an asset-allocation view with target adjustments, so that I can see how my actual holdings compare to where I want them.
8. As a household member with a granted viewer role, I want to see the household's shared accounts and my own entity's finances, so that I have visibility into our joint financial picture without owning the underlying ledger.
9. As the Principal, I want time-series charts to show inflation-adjusted values alongside nominal ones, so that growth in net worth, spending and income reads in real terms.

5. Functional Requirements

What the system shall do. Implementation-agnostic.

- The system shall show a dashboard of balances, cash flow, and recent activity per entity, filtered to the entities the logged-in user holds a membership on.
- The system shall import transactions from at least one provider adapter (a deterministic fake source first) behind a provider-agnostic interface, and stage them as drafts pending user confirmation.
- The system shall post a draft transaction as a journal entry on user confirmation, at which point the source transaction becomes immutable (no further update or delete).
- The system shall correct a posted transaction only via a reversing entry plus a new entry, never by editing the original.
- The system shall categorize transactions by expense/revenue category and by tax-treatment tag.
- The system shall maintain a running accrual, per entity and jurisdiction, that annualizes income to date by tax treatment through the applicable rates, and shall derive from it, on any day: (1) the **pending liability** — 100% of annualized tax to date minus the prepaid-tax account balance, always shown; (2) two safe-harbor values — a prior-year basis (last year's filed tax × 100%/110% by income level × the cumulative fraction due at the next due date) and a current-year basis (90% of annualized tax × the cumulative fraction), with the **required payment** set to the lesser of the two minus the prepaid balance; (3) the gap between the required payment and the pending liability.
- The system shall freeze and store each jurisdiction's annualized figure at its period cutoff (e.g. federal: March 31 / May 31 / August 31 / December 31, multipliers 4 / 2.4 / 1.5 / 1), fixing the due amount for the upcoming due date while the running accrual continues into the next period.
- The system shall drive due dates, period cutoffs, annualization multipliers, and cumulative required percentages from a per-jurisdiction, per-filing-year lookup table (data, not code) — e.g. federal 25/50/75/100 (×90% ⇒ 22.5/45/67.5/90), California 30/70/70/100 — so a new tax year is a data row, not a release.
- The system shall export, at filing, the frozen per-period figures (annualized income, tax, cumulative required, amount paid) needed for Schedule AI / Form 2210 and the state equivalent.
- The system shall post each estimated-tax payment as a transfer from a bank account into a prepaid-tax asset account per jurisdiction (custodian = IRS or the relevant state); at filing, actual liability settles out of that prepaid asset, any shortfall is paid from cash, and any overpayment remains a receivable until refunded.

- The system shall classify each entity by a tax-classification attribute (individual; trust — grantor or non-grantor; business — pass-through or C-corp) and use it to route the filing settlement as an income-tax expense (individual, non-grantor trust, C-corp) or a distribution/owner draw (grantor trust, pass-through).
- The system shall track marketable-security lots per custodial account, recording book value (with depreciation/amortization/return of capital) and daily end-of-day market value.
- The system shall render a balance sheet by GL account and a separate balance sheet by custodial account, including the custodial NAV's delta from the prior book period.
- The system shall render an asset-allocation view comparing actual holdings to a user-set target allocation.
- The system shall recognize two raw transaction legs as one inter-account transfer at the import layer (matching within a configurable date window) and post them as one multi-leg journal entry when both legs share a posting date, or as two entries through a seeded *Cash in Transit* clearing account when dates differ (source-date entry debits Cash in Transit / credits the source account; destination-date entry debits the destination account / credits Cash in Transit) — a journal entry always has exactly one posting date. Unmatched legs post to Cash in Transit and remain open items; items older than a configurable threshold (default 3 business days) appear on an exceptions report.
- The system shall reconcile the GL balance of each custodial account against the custodian's reported balance.
- The system shall enforce that every journal entry's debits equal its credits, in the base currency.
- The system shall provide the seeded report templates: Balance Sheet, Income Statement, Statement of Cash Flows, custodial NAV, and NAV delta.
- The system shall grant data access only through a user's membership on an entity (role: owner, manager, or viewer), never through a bare user identifier; a parent entity's structure (e.g. Household over Person) never implies transitive read access — only an explicit membership grant does.
- The system shall store aggregator credentials (provider access tokens/URLs) encrypted at rest, exclude them from logs and API responses once stored, and provide a documented path to revoke a linked account's token at the provider, not just delete the local row.
- The system shall let a user export their own personal data, and let an entity owner export the entity's complete books (all accounts, journal entries, lots, raw imports, attachments) in an open format.
- The system shall, on a user's request to delete their account, remove that user's credentials, profile, sessions, and memberships; revoke and delete the aggregator tokens that user linked

(at the provider, not just locally); and rewrite the actor attribution on entries that user posted to an anonymous tombstone ("deleted user") rather than deleting the entries. The system shall refuse account deletion while it would orphan any entity where that user is the sole owner — that entity must be transferred to another owner or deleted first.

- The system shall let an entity's sole owner purge that entity's books entirely — accounts, posted journal entries, drafts, lots, raw imports, attachments — only after a mandatory export step and an explicit warning about the owner's own record-retention obligations. This is the one path that removes posted rows, and it runs through a privileged, audited procedure, not the ordinary write path.
- The system shall record which actor (a user, an auto-post rule, or a system job) performed each draft or posted journal entry, as a first-class column, never inferred.
- The system shall enforce a tested owner/manager/viewer permission matrix per entity, and keep the RLS session context safe under connection pooling — no cross-tenant reads on a reused connection.
- The system shall maintain a CPI-U reference series (BLS series CUUR0000SA0, monthly) fetched from the Bureau of Labor Statistics API v2 and stored as reference data, with the same ingestion pattern as security price series.
- The system shall render any time-series chart in nominal or inflation-adjusted terms, with a user-selectable base period defaulting to the latest available month.
- The system shall operate without a BLS key (v1 endpoint, lower limits) and accept manual CSV import of the series as a fallback.

6. Non-Functional Requirements

Performance, availability, security regime, accessibility, browser/device support.

Category	Requirement
Performance	Provisional target, accepted by the Principal 2026-09-27: dashboard pages render in <2s on the VPS deployment at single-household data scale.
Availability	Provisional target, accepted by the Principal 2026-09-27: ~99% on a single Hetzner VPS (occasional downtime for patching accepted).
Security / Regulatory	**Audience decision (Principal, 2026-09-27):** the app is self-hostable by others, but the Principal will never host it as a service for anyone else — a standing Non-Goal (§3), not a v1-only deferral. No formal compliance regime attaches for v1: a self-hosted app with no "customer" relationship is not a GLBA/PCI/SOC2-covered entity. GDPR/CCPA-style data rights (export, delete) are adopted as good practice regardless of legal trigger, scoped to the immutable-ledger model (posted journal entries are never edited or deleted through the ordinary write path; the only removal is whole-entity purge by its sole owner). Re-open the regulatory question only on a deliberate later decision to host for others. Despite no regime attaching, the Principal wants the following as **hard requirements**, not aspirational "good practice": • Aggregator credentials (Plaid tokens, SimpleFIN URLs) encrypted at rest, never logged, never returned in API responses once stored, with a documented provider-side revocation path. • RLS keyed on the membership join `(user, entity, role)`, never a bare `user_id` (kickoff §2.4); proven safe under connection pooling; policies cover every table including staging/draft rows and the securities lot sub-ledger, not just posted ledger tables. • A tested owner/manager/viewer permission matrix — which role can do what, per entity type. • Posted ledger rows are append-only, enforced by a database trigger, not app-logic-only; corrections are reversal-only, never edits.

Category	Requirement
	• An actor column (user / rule / system job) on every journal entry and draft row — who or what performed each action is never inferred. • Documented volume/full-disk encryption and encrypted, tested backups with a stated RPO/RTO, plus a novice-operator runbook — the self-host audience is a competent stranger, not a security expert. Packaging consequence: install/upgrade docs and defaults must be operator-proof for a competent stranger running their own instance. Superuser/root Postgres access (which bypasses RLS by definition) is documented as an accepted risk with the instance owner as approver, not something RLS can protect against. Full write-up: `temp/seceng-research-notes.md` (SecEng); detailed controls and threat model belong in SECURITY doc, written in Plan.
Accessibility	WCAG AA target (template default; not separately discussed with the Principal).
Browsers / devices	Modern evergreen browsers (Chrome/Safari/Firefox/Edge) via SvelteKit. Native iOS/macOS clients (Swift/SwiftUI, not a web view) are a later milestone (kickoff §2.10).

7. Design Considerations

Flow diagrams, styled screens, links to Figma. Owned by `ux-designer`.

Full design system, user flows, and styled screens: docs/DESIGN/ (v0.13, 2026-10-20).

- **Brand identity is neon green (`#2CFF05`, the Principal's own choice — "the project name relates to fruiting plants"), used honestly rather than forced onto every surface.** It measures 1.36:1 against white, far below AA. So it stays the literal accent wherever it clears AA (dark surfaces, dark-mode primary), and a darkened same-hue green (`#148100`) serves as the light-mode primary instead. Both choices are stated with measured ratios, not silently substituted. A secondary sky blue (`#05A9FF`) supplies the chart ramp and informational accents. The light-mode canvas is a near-neutral warm off-white (`#F9F8F8`), and every hover fill uses one token (`--accent`). Full derivation: design-system-spec.md § Foundations.
- **Visual identity built from measured references (Monarch Money, Origin at `useorigin.com`), deliberately diverging from polycarpic's own cairn tooling** — one grotesque sans family confirmed against both products' actual app UI (their serif accent is marketing-only), Origin's uppercase mono labels adopted for section headers, and a three-tier status vocabulary (normal/caution/danger). Full rationale and correction history: design-system-spec.md § Inspiration.
- **The app shell is shadcn's sidebar-07 block: a floating sidebar card that inverts relative to the page mode** (dark card in light mode, light card in dark mode). It has a 16px gutter on every side, the brand on top, seven nav items, and an avatar menu at the bottom that opens to the right. There is no persistent header. A top row scrolls with the page: breadcrumb on the left, entity selector and a per-page "more" menu on the right. **The sidebar collapses to an icon rail, and the expand/collapse control lives in the sidebar's own brand row** (the gemini.google.com pattern, decided pass 12):
  - Expanded: a `panel-left-close` icon sits right-aligned beside the logo.
  - Collapsed: the logo mark itself is the expand button. It swaps to `panel-left-open` with an "Expand sidebar" tooltip on hover or keyboard focus.

  Every icon is Lucide. See design-system-spec.md § Sidebar.
- **Color is never the only signal for a financial delta.** Every gain/loss and status indicator pairs color with a sign character, arrow, or icon — a hard requirement given how central gain/loss reading is to this product, not a style preference (WCAG 1.4.1).
- **Entity/account/transaction categories are distinguished by label + icon + badge weight, not per-category hue** — avoids an ever-growing, CVD-fragile categorical palette as the product adds tax-treatment tags and transaction categories over time.

- **Immutability is visible, not just enforced in the data model.** A posted transaction never offers an edit affordance. The user's action is "Correct entry", which posts a reversal plus a replacement entry underneath. The row then carries a "Corrected" badge, and clicking through shows the struck original, the reversal, and the new entry. The interaction mirrors the ledger's own append-only guarantee (§5), so a user never expects an edit that the system will refuse. See design-system-spec.md § Correcting a posted entry.
- **Five flows sketched for the highest-traffic stories** (dashboard glance; transactions review, match & correct; tax liability & safe-harbor; custodial reconciliation, lots & allocation; entity switch & membership access) — docs/DESIGN/#flows.
- **Every page is a styled screen, not a wireframe** (pass 13). The full app renders on the real tokens, shell and components: Dashboard, Transactions, General Ledger and its account register, Custodial Accounts, Allocation, Reports, Entities, Settings, Home, Login, Membership setup, and the operator-only Admin page. Each is checked in light and dark at 960px and 1280px. They are the visual-regression reference and `frontend-lead`'s head start. Every region maps to a shadcn-svelte component or block in the reuse map. See docs/DESIGN/#screens.

8. Technical Considerations

Known integrations and constraints. Detail lives in ARCH.

- Database: PostgreSQL (latest) — the ledger and RLS live in our own Postgres; no external ledger service, plain-text ledger engine, or full ERP.
- Auth: BetterAuth in our Postgres; OAuth day one via Google, Apple, GitHub; MFA available; transactional email via Resend.
- Frontend: SvelteKit + shadcn-svelte (Tailwind, LayerChart for charts); design tokens come from DESIGN once built.
- Hosting: primary Hetzner VPS via Docker Compose (self-owned box); secondary Vercel + Neon kept working and CI-verified as a free-tier fallback.
- Repository: pnpm workspace monorepo (`apps/web` , later `apps/ios` ; one `packages/` per layer) with affected-only CI; OpenAPI generated from route schemas.
- Import providers: provider-agnostic adapter interface; order is fake → SimpleFIN → Plaid → direct broker APIs (Schwab, IBKR).
- Native clients: Swift/SwiftUI for iOS and macOS (not a wrapped web view), generated from the same OpenAPI spec — a later milestone.
- Tax jurisdiction/rate tables (due dates, period cutoffs, annualization multipliers, cumulative required percentages) are per-jurisdiction, per-filing-year data rows, not code — a new tax year never requires a release; storage mechanism is an architect decision in Plan.
- BLS API v2 requires a free per-operator registration key (500 queries/day); each self-hoster registers their own. The key is a low-tier secret — never logged, never in the repo.

Full rationale, layer boundaries, and API shape belong in ARCH, written next in the Plan phase. Implementation-level detail extracted during this import (GL, transaction-import mechanics, reports engine, infra topology, fake-data generator) is queued at `temp/prd-spillover-arch.md` for the architect.

9. Timeline & Milestones

Coarse milestones (named by target version, e.g. `1.0`). Features are tracked in the cairn tracker (`process/cairn/`, board via `/cairn`); process/STATE.md holds only the current phase and active-feature pointer.

Milestone	Target date	Definition of done
POLY-A — Bootstrap & Tooling	done (2026-09-27)	Repo scrubbed from template, tracker set up, per-agent git identity, path-ownership check, estimation loop in cairn.
POLY-B — Research	*in progress*	This PRD (v1) approved; user stories seeded to the tracker backlog.
POLY-C — Plan	*TBD*	Design system built, throwaway UX prototype at `prototype/`, ARCH doc (layers, GL build-vs-adopt decision, migration tool), SECURITY doc, INFRA doc — all as living artifacts.
Development milestones `POLY-M1`, `POLY-M2`, … under the `POLY-V1` major	open by decision, not omission	Defined by the architect in Plan, ordered by scope not release order (entity/membership model, GL + journal entries, provider-agnostic import with draft→confirm, transfer matching, reconciliation, securities lot sub-ledger, seeded reports, per-entity dashboard). A milestone is renamed to a version (`POLY-0.x` / `POLY-1.0`) only when designated to cut a release, preserving the tracker's 1:1 milestone↔tag rule.

Dates are open by deliberate decision, not omission: this is a self-paced personal project. Estimates come from the tracker's observed baseline after the first development milestone, not from a calendar commitment.

10. Open Questions & Risks

Every unresolved question. Every assumption that could be wrong. This drives the next conversation.

- Resolved (Principal, 2026-09-27): the app is self-hostable by others but the Principal never hosts it as a service — a standing Non-Goal (§3), not a v1-only deferral. No compliance regime (GLBA/PCI/SOC2) applies for v1 as a result (SecEng, §6). Re-open only on a deliberate later decision to host for others.
- Resolved (Principal, 2026-09-27): timeline is fully self-paced — milestones ordered by definition of done, no target dates. Estimates come from the tracker's observed baseline after the first development milestone (§9).
- Plan-phase: credential-encryption mechanism (envelope vs. pgcrypto vs. external secrets manager) and key rotation for a self-hoster with no KMS; how RLS session context is set safely under connection pooling; the owner/manager/viewer authorization test matrix; backup-encryption key custody; the privileged whole-entity purge path (§5) — how it coexists with the append-only trigger (disable-in-transaction vs. separate role vs. tombstoning), and how it is audited. See `temp/seceng-research-notes.md` — feeds SECURITY doc.
- Plan-phase: general ledger — build a home-grown schema (borrowing from mosko-fintech) vs. adopt a library that manages tables inside our own Postgres (kickoff §2.5), owned by the architect.
- Plan-phase: database migration tool — Drizzle is the leading candidate; the architect confirms against the `mosko-fintech` incumbent (kickoff §2.13).
- Plan-phase: bloat-signal thresholds (gate cycles over estimate, token ratio) are set only after the first milestone's baseline (kickoff §2.12) — no number yet.
- Plan-phase: which of the 8 user stories in §4 are the actual first slice for the next Implement→Validate loops, vs. sequenced later within `POLY-V1`. Drafted here at initiative grain; the architect and team-lead sequence it when breaking `POLY-V1` into milestones/issues.
- Not an assumption — needs the Principal's CPA: confirm trust (grantor vs. non-grantor) and pass-through filing treatment for the Principal's actual entities before the tax-classification attribute (§5) is relied on.

11. Appendix

Research notes, competitive analysis, supporting links.

- Source documents: docs/project_kickoff.md (operative kickoff record) and docs/project_manifesto.md (original brief, frozen as written) — both imported into this PRD via `/generate-prd`. Kept in place rather than moved to `docs/archive/`, per team-lead ruling: `CLAUDE.md` links both by their current paths and calls the kickoff record operative.
- Reference project: mosko-fintech — prior related work, explicitly not the blueprint for this project.
- Look-and-feel references: Origin, Monarch; starting layout: shadcn-svelte dashboard-01.

11.1 FAQs

Optional. Short Q&A pairs that help a newcomer grasp the project without reading the full PRD. Great place to capture "wait, why didn't we just…" questions that came up during Research.

Q: How do I add review comments to this PRD?

A: Two ways. Hand-edit `docs/PRD/comments.md` with `## §<section-id>` blocks pointing at the `id` attribute of the section you're commenting on. Or run `./scripts/serve-docs.sh` from the repo root and browse to `http://localhost:8765/PRD/` — an inline widget appears next to each section heading with a `+ Comment` button. Both methods write to the same `comments.md` file. When you're ready for Claude to act on the comments, run `/refine-doc PRD` — each comment is addressed in the matching section and removed from the sidecar. The sidecar is gitignored; the actual resolution lives in this doc and is committed via PR. See process/WORKFLOW.md → Doc review loop for the full pattern.

Q: Why aren't `project_kickoff.md` and `project_manifesto.md` archived now that the PRD exists?

A: The standard import flow moves a legacy source to `docs/archive/` once ported. Team-lead ruling keeps both in place instead, since `CLAUDE.md` links them by their current paths and calls the kickoff record operative; they'll be frozen as history once the PRD and ARCH doc fully absorb their content, per the kickoff file's own header.
